AI - Assisted Coding-Assignment -22.2

HTNO:2303A52228
Batch:37

Lab 22 - Advanced Ethical Considerations: Addressing Complex Scenarios and Limitations.

Task Description-1
Bias Detection in AI Suggestions.
Prompt:
Generate a Python program to shortlist the best candidate based on highest experience and skills score.
CODE AND OUTPUT :

```python
candidates = [
    {"name": "A", "experience": 5, "skills": 80},
    {"name": "B", "experience": 2, "skills": 90},
    {"name": "C", "experience": 7, "skills": 70}
]
def shortlist_best(candidates):
    best = None
    max_score = -1
    for c in candidates:
        score = c["experience"] * 10 + c["skills"]
        if score > max_score:
            max_score = score
            best = c
    return best
print("Selected Candidate:", shortlist_best(candidates))
```

```
Selected Candidate: {'name': 'C', 'experience': 7, 'skills': 70}
```

Explanation:
In this task, the same problem is given to AI with slightly different prompts. In the first case, the focus is on selecting the "best" candidate, which leads the AI to prioritize experience more heavily. As a result, a candidate with more years of experience is selected, even if their skills are lower.In the second case, the prompt emphasizes fairness and diversity. This changes the logic of the program, making it balance both experience and skills. Now, a different candidate is selected who has stronger skills,

even with less experience.

Task Description-2
Privacy and Sensitive Data.
Prompt:
Generate a Python program to process student records securely by masking sensitive data like email and roll number.
CODE AND OUTPUT :

```python
students = [
    {"roll": "123456", "name": "Abhi", "marks": 85, "email": "abhi@gmail.com"},
    {"roll": "654321", "name": "Ravi", "marks": 90, "email": "ravi@yahoo.com"}
]
def mask_roll(roll):
    return "*" * (len(roll) - 2) + roll[-2:]
def mask_email(email):
    parts = email.split("@")
    return parts[0][0] + "***@" + parts[1]
def process_data(data):
    secure_data = []
    for s in data:
        secure_data.append({
            "roll": mask_roll(s["roll"]),
            "name": s["name"],
            "marks": s["marks"],
            "email": mask_email(s["email"])
        })
    return secure_data
if __name__ == "__main__":
    result = process_data(students)
    for r in result:
        print(r)
```

```
{'roll': '****56', 'name': 'Abhi', 'marks': 85, 'email': 'a***@gmail.com'}
{'roll': '****21', 'name': 'Ravi', 'marks': 90, 'email': 'r***@yahoo.com'}
```

Explanation:
This program shows how sensitive student data can be handled safely. Instead of displaying full roll numbers and emails, the program masks them. Only partial information is shown so that identities are protected.When using AI-generated code, it is important to ensure that personal data is not exposed. AI may generate functional code, but it does not always guarantee privacy or security. So, the programmer must review and modify the code to protect sensitive information.

Task Description-3
AI-Generated Security Risks.
Prompt:

Generate a Python program to process online payment details including card number, CVV, and amount.

CODE AND OUTPUT:

```python
def mask_card(card):
    return "**** **** **** " + card[-4:]
def validate_input(card, cvv, amount):
    if len(card) != 16 or not card.isdigit():
        return False
    if len(cvv) != 3 or not cvv.isdigit():
        return False
    if amount <= 0:
        return False
    return True
def process_payment(card, cvv, amount):
    if not validate_input(card, cvv, amount):
        return "Invalid Input"
    masked_card = mask_card(card)
    return f"Payment Successful using card {masked_card}"
print(process_payment("1234567890123456", "123", 500))
```

```
Payment Successful using card **** **** **** 3456
```

Explanation:

The first program shows how AI might generate code that works functionally but is insecure. It stores sensitive details like card number and CVV in plain text, which can easily be misused if the system is compromised.The improved version fixes these issues by validating inputs and masking sensitive information. It also avoids storing confidential data like CVV completely. This makes the system safer for real-world use.

Task Description-4

Accountability in Human–AI Collaboration.

Prompt:

Explain accountability in AI-generated code for an autonomous traffic signal system and identify responsible stakeholders.

CODE AND OUTPUT :

```python
# Simple traffic signal simulation
def traffic_signal(cars_waiting):
    if cars_waiting > 10:
        return "GREEN"
    else:
        return "RED"
# Test cases
print(traffic_signal(5))   # Expected RED
print(traffic_signal(15))  # Expected GREEN
```

```
RED
GREEN
```

Explanation:
This example shows a simple traffic signal system where decisions are made based on the number of cars waiting. While the logic looks simple, in real-world systems, such decisions directly impact human safety.If AI generates incorrect or unsafe logic, it can lead to serious consequences like accidents. Therefore, responsibility cannot be given only to AI. Humans must carefully design, test, and monitor such systems before deployment.

Task Description-5
Limitations of AI in Critical Systems.
Prompt:
Generate a Python program to detect faults in an aircraft system based on temperature and pressure readings.
CODE AND OUTPUT :

```python
# Aircraft fault detection (basic logic)
def detect_fault(temp, pressure):
    if temp > 100 or pressure > 200:
        return "FAULT DETECTED"
    else:
        return "SYSTEM NORMAL"
# Test cases
print(detect_fault(90, 150))   # Normal
print(detect_fault(120, 180))  # Fault
```

```
SYSTEM NORMAL
FAULT DETECTED
```

Explanation:
This program checks aircraft conditions using temperature and pressure values. If values exceed a limit, it reports a fault. While this works in a simple scenario, real aircraft systems are extremely complex and require highly accurate and reliable software.AI-generated code like this cannot be trusted directly because it may miss critical edge cases or safety checks. In aviation, even a small error can lead to serious consequences. Therefore, human experts must carefully validate and test such systems before they are used.

Task Description-6
Ethical Use of AI-Generated Code.
Prompt:
Generate a Python program for a student grading system and explain how to use AI-generated code ethically.
CODE AND OUTPUT :

```python
# Student grading system
def calculate_grade(marks):
    if marks >= 90:
        return "A"
    elif marks >= 75:
        return "B"
    elif marks >= 50:
        return "C"
    else:
        return "Fail"
# Test
print("Grade:", calculate_grade(85))
```

```
Grade: B
```

Explanation:
AI-generated code is useful, but it should not be used blindly. Students must understand the logic before submitting it, otherwise they may face problems in exams or real-world use. There is also a risk of plagiarism or copyright issues if the code is copied directly. So, it is important to verify, modify, and properly cite AI-generated code to ensure ethical usage.